Data Structure

Unit-I: Introduction of Data Structure

Data Structure: A data structure is a specialized format for organizing, processing, retrieving and storing data.

Algorithm*:* A set of rules to be followed in calculations or other problem-solving operations*" Or "*A procedure for solving a mathematical problem in a finite number of steps that frequently involves recursive operations*.*

- **Characteristics Algorithm**

There are some characteristics which every algorithm should follow. There are six different characteristics which deal with various aspects of algorithm. They are as follows:

1.Input specified
2.Output specified
3.Definiteness
4.Effectiveness
5. Finiteness
6.Independent

Let us see these characteristics one by one.

1)Input specified

The input is the data to be transformed during the computation to produce the output. An algorithm should have 0 or more well-defined inputs. Input precision requires that you know what kind of data, how much and what form the data should be.

2)Output specified

The output is the data resulting from the computation (your intended result). An algorithm should have 1 or more well-defined outputs, and should match the desired output. Output precision also requires that you know what kind of data, how much and what form the output should be (or even if there will ***be*** any output at all!).

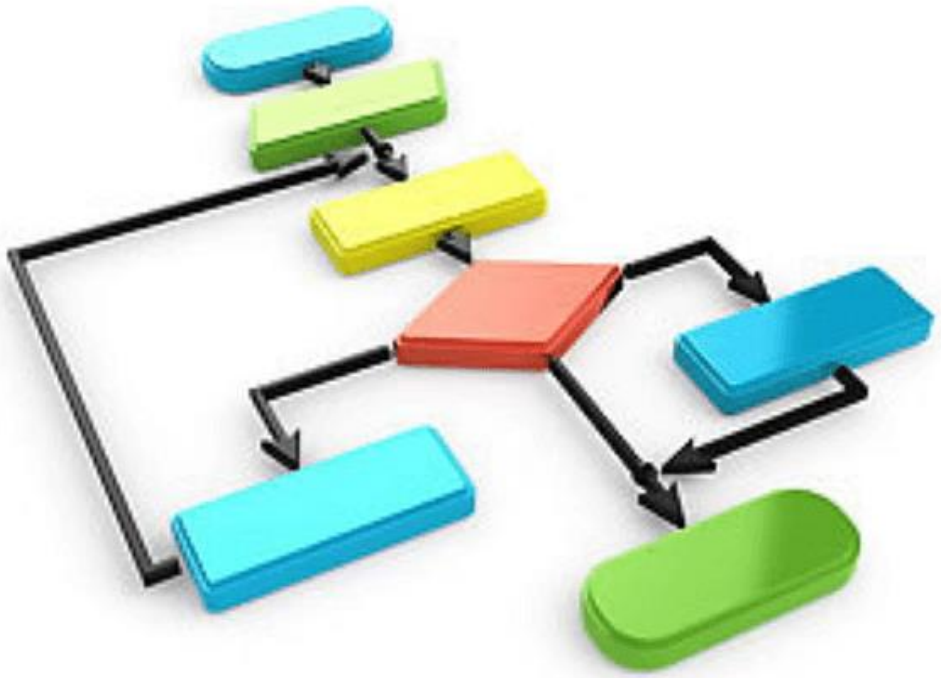
3)Definiteness

Algorithms must specify every step and the order the steps must be taken in the process. Definiteness means specifying the sequence of operations for turning input into output. Algorithm should be clear and unambiguous. Details of each step must be also be spelled out (including how to handle errors). It should contain everything quantitative and not qualitative.

4)Effectiveness

For an algorithm to be effective, it means that all those steps that are required to get to output must be feasible with the available resources. It should not contain any unnecessary and redundant steps which could make an algorithm ineffective.

For example, suppose you are cooking a recipe and you chop vegetables which are not be used in the recipe then it is a waste of time.



5)Finiteness

The algorithm must stop, eventually. Stopping may mean that you get the expected output OR you get a response that no solution is possible. Algorithms must terminate after a finite number of steps. An algorithm should not be infinite and always terminate after definite number of steps.

There is no point in developing an algorithm which is infinite as it will be useless for us.

6)Independent

An algorithm should have step-by-step directions, which should be independent of any programming code. It should be such that it could be run on any of the programming languages.

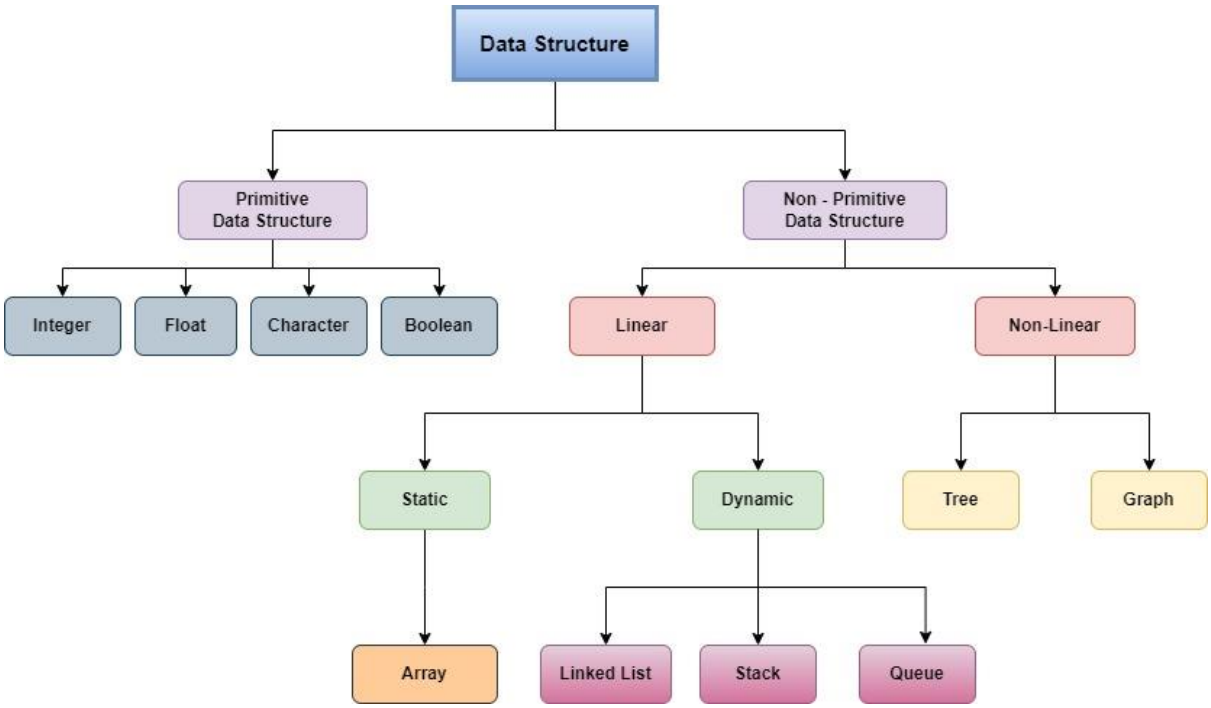
- **Need of Data Structure**

As applications are becoming more complex and the amount of data is increasing day by day, which may cause problems with processing speed, searching data, handling multiple requests etc. Data structure provides a way of organizing, managing, and storing data efficiently. With the help of data structure, the data items can be traversed easily. Data structure provides efficiency, reusability and abstraction. It plays an important role in enhancing the performance of a program because the main function of the program is to store and retrieve the user's data as fast as possible.

Why Learn Data Structure?

Data structure and algorithms are two of the most important aspects of computer science. Data structures allow us to organize and store data, while algorithms allow us to process that data in a meaningful way. Learning data structure and algorithms will help you become a better programmer. You will be able to write code that is more efficient and more reliable. You will also be able to solve problems more quickly and more effectively.

❖ Types of Data Structure



There are 2 types of Data Structure:

- Primitive Data Structure
- Non – Primitive Data Structure

Primitive Data Structure –

Primitive Data Structures directly operate according to the machine instructions. These are the primitive data types. Data types like int, char, float, double, and pointer are primitive data structures that can hold a single value.

Non – Primitive Data Structure –

Non-primitive data structures are complex data structures that are derived from primitive data structures. Non – Primitive data types are further divided into two categories.

- Linear Data Structure
- Non – Linear Data Structure

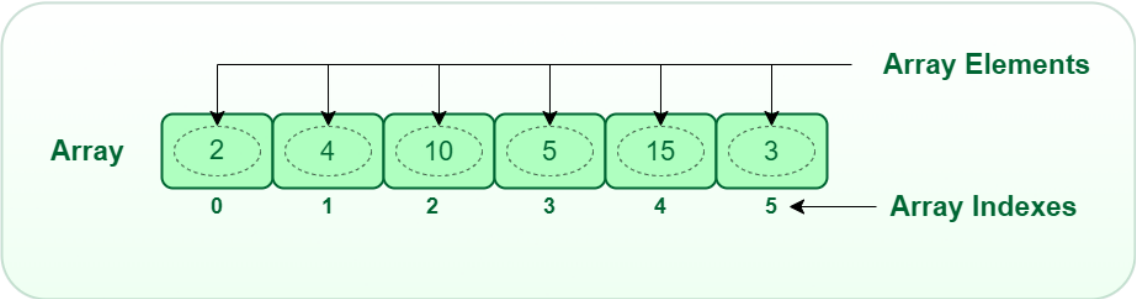
Linear Data Structure –

Linear Data Structure consists of data elements arranged in a sequential manner where every element is connected to its previous and next elements. This connection helps to traverse a linear arrangement in a single level and in a single run. Such data structures are easy to implement as memory is additionally sequential. Some examples of Linear Data Structure are List, Queue, Stack, Array etc.

Types of Linear Data Structure –

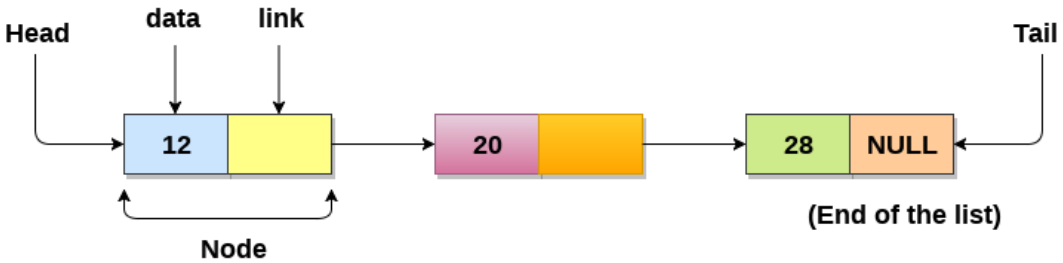
1] Arrays –

An array is a collection of similar data elements stored at contiguous memory locations. It is the simplest data structure where each data element can be accessed directly by only using its index number.



2] Linked List –

- A linked list is a linear data structure that is used to maintain a list-like structure in the computer memory.
- Linked List can be defined as collection of objects called **nodes** that are randomly stored in the memory.
- A node contains two fields i.e. data stored at that particular address and the pointer which contains the address of the next node in the memory.
- The last node of the list contains pointer to the null.

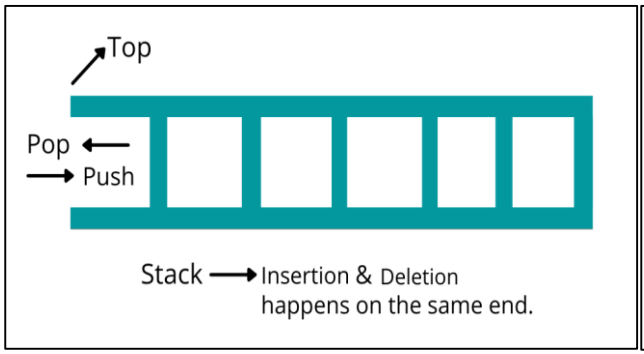


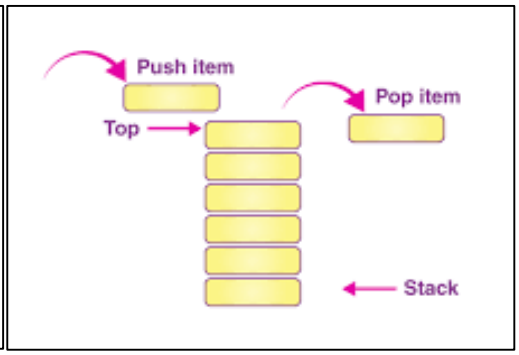
3] Stack –

Stack is a linear data structure that follows a specific order during which the operations are performed. The order could be FILO (First In Last Out) or LIFO (Last In First Out).

The basic operations performed in stack are as follows:

- Push – Adds an item within the stack.
- Pop – Deletes or removes an item from the stack.
- Top – Returns the topmost element of the stack.
- IsEmpty – Returns true if the stack is empty.





4] Queue –

Queue is a linear data structure in which elements can be inserted from only one end which is known as rear and deleted from another end known as front. It follows the FIFO (First In First Out) order.

- **Enqueue**: Add an element to the end of the queue
- **Dequeue**: Remove an element from the front of the queue
- **IsEmpty**: Check if the queue is empty
- **IsFull**: Check if the queue is full

Queue Data Structure

Non-Linear Data Structure:
Non-linear Data Structures do not have any set sequence of connecting all its elements and every element can have multiple paths to attach to other elements. Such data structures support multi-level storage and sometimes can't be traversed in a single run. Such data structures aren't easy to implement but are more efficient in utilizing memory. Some examples of non-linear data structures are Tree, BST, Graphs etc.

Types of Non-Linear Data Structure

1] Tree –

A tree is a multilevel data structure defined as a set of nodes. The topmost node is named root node while the bottom most nodes are called leaf nodes. Each node has only one parent but can have multiple children.

Types of Trees in Data structure

- General Tree
- Binary Tree
- Binary Search Tree
- AVL Tree
- Red Black Tree
- N-ary Tree

2] Graph

A graph is a pictorial representation of a set of objects connected by links known as edges. The interconnected nodes are represented by points named vertices, and the links that connect the vertices are called edges.

Types of Graph

- Finite Graph
- Infinite Graph
- Trivial Graph
- Simple Graph
- Multi Graph
- Null Graph
- Complete Graph
- Pseudo Graph
- Regular Graph
- Bipartite Graph
- Labeled Graph
- Diggraph Graph
- Subgraph
- Connected or Disconnected Graph
- Cyclic Graph

- Vertex Labelled Graph
- Directed Acyclic Graph

❖ Various Data Structures Operations

Data structure operations are the methods used to manipulate the data in a data structure. The most common data structure operations are:

- **Traversal**

  Traversal operations are used to visit each node in a data structure in a specific order. This technique is typically employed for printing, searching, displaying, and reading the data stored in a data structure.

- **Insertion**

  Insertion operations add new data elements to a data structure. You can do this at the data structure's beginning, middle, or end.

- **Deletion**

  Deletion operations remove data elements from a data structure. These operations are typically performed on nodes that are no longer needed.

- **Search**

  Search operations are used to find a specific data element in a data structure. These operations typically employ a compare function to determine if two data elements are equal.

- **Sort**

  Sort operations are used to arrange the data elements in a data structure in a specific order. This can be done using various sorting algorithms, such as insertion sort, bubble sort, merge sort, and quick sort.

- **Merge**

  Merge operations are used to combine two data structures into one. This operation is typically used when two data structures need to be combined into a single structure.

- **Copy**

  Copy operations are used to create a duplicate of a data structure. This can be done by copying each element in the original data structure to the new one.

❖ Time Complexity

The time complexity of an algorithm refers to the amount of time required by the algorithm to execute and get the result. This can be for normal operations, conditional if-else statements, loop statements, etc.

How to Calculate, Time Complexity?

The time complexity of an algorithm is also calculated by determining the following 2 components:

- **Constant time part:** Any instruction that is executed just once comes in this part. For example, input, output, if-else, switch, arithmetic operations, etc.
- **Variable Time Part:** Any instruction that is executed more than once, say n times, comes in this part. For example, loops, recursion, etc.
  Therefore Time complexity of any algorithm P is **T(P) = C + TP(I)**, where C is the constant time part and TP(I) is the variable part of the algorithm, which depends on the instance characteristic I.

Example: In the algorithm of Linear Search above, the time complexity is calculated as follows:

Step 1: –Constant Time
Step 2: — Variable Time (Taking n inputs)
Step 3: –Variable Time (Till the length of the Array (n) or the index of the found element)
Step 4: –Constant Time
Step 5: –Constant Time
Step 6: –Constant Time
Hence, T(P) = 1 + n + n(1 + 1) + 1 = 2 + 3n, which can be said as T(n).

❖ Space Complexity

The space complexity of an algorithm refers to the amount of memory required by the algorithm to store the variables and get the result. This can be for inputs, temporary operations, or outputs.

How to calculate Space Complexity?

The space complexity of an algorithm is calculated by determining the following 2 components:

- **Fixed Part:** This refers to the space that is required by the algorithm. For example, input variables, output variables, program size, etc.
- **Variable Part:** This refers to the space that can be different based on the implementation of the algorithm. For example, temporary variables, dynamic memory allocation, recursion stack space, etc.
  Therefore Space complexity **S(P)** of any algorithm P is **S(P) = C + SP(I)**, where C is the fixed part and S(I) is the variable part of the algorithm, which depends on instance characteristic I.

Example: Consider the below algorithm for Linear Search

Step 1: START
Step 2: Get n elements of the array in arr and the number to be searched in x
Step 3: Start from the leftmost element of arr[] and one by one compare x with each element of arr[]
Step 4: If x matches with an element, Print True.

Step 5: If x doesn't match with any of the elements, Print False.
Step 6: END
Here, There are 2 variables arr[], and x, where the arr[] is the variable part of n elements and x is the fixed part. Hence S(P) = 1+n.

So, the space complexity depends on n(number of elements). Now, space depends on data types of given variables and constant types and it will be multiplied accordingly.

❖ Applications of Data Structure

Data structures is the core of software development, any efficient method to a given problem is contingent on how effectively data is arranged. Hash tables are used to build identifier lookups in compiler implementations. The B-tree data structures are appropriate for use in databases.

The following are some of the most important areas where data structures are used:

1. **Artificial Intelligence:** Data structures are vital for managing and optimizing large volumes of data in AI applications, facilitating efficient pattern recognition, decision-making, and problem-solving algorithms.
2. **Compiler Design:** Data structures like hash tables play a crucial role in compiler implementations, enabling efficient identifier lookups and symbol table management during code compilation.
3. **Machine Learning:** Data structures are essential for organizing and processing training datasets, enabling machine learning algorithms to classify, predict, and learn from data effectively.
4. **Database Design and Management:** Databases rely on data structures like B-trees to ensure efficient data retrieval, insertion, and modification, maintaining the integrity and performance of database systems.
5. **Blockchain:** Data structures underpin the decentralized nature of blockchain technology, helping record and secure transaction data in a tamper-resistant and distributed ledger format.
6. **Numerical and Statistical Analysis:** Data structures support numerical and statistical analysis by organizing data for various computational tasks, such as simulations, optimization, and data-driven insights.
7. **Operating System Development:** Data structures are fundamental components in OS development, providing efficient memory management, process scheduling, file system organization, and interprocess communication.
8. **Image & Speech Processing:** Data structures assist in representing, processing, and analysing multimedia data, contributing to applications like image recognition, speech synthesis, and digital signal processing.
9. **Cryptography:** Cryptographic algorithms rely on data structures to securely store keys and manage encrypted data, ensuring the confidentiality and integrity of sensitive information.

****